Summary “Generic Schema Matching with Cupid”

By Ossama Sijbesma (9890238) – 20 October 2026

Schema matching is an operation that takes two schemas, such as the ones in figure 1, and returns a mapping from one element to another based on a similarity match. In figure 1 a mapping might pair *Lines.Item.Qty* with *Items.Item.Quantity*. Schema matching is used in many applications where there is a need for such mappings. From a professional point of view, I could see this being used for related data discovery within large data platforms. Within the paper they also mention mapping messages across different file formats (XML, Json, etc.), consolidating different data sources into a unified data warehouse, and identifying integration points between heterogeneous databases for mediators.

The main problem

At the time that the paper was written, schema matching was primarily performed as a manual task by domain experts. There was some automated tooling, but all it could do was match schema elements with the same name and same structure.

PO	POrder
Lines	Items
Item	Item
Line	ItemNumber
Qty	Quantity
Uom	UnitOfMeasure

Figure 1, Two schemas to be matched

The authors also mention that schema matching is tricky. Even when two schemas deal with the same concept, they often have different structures and different names (see figure 1). There is also the issue that the same word can have different meanings depending on the context.

Furthermore, schema matching is subjective. Schemas don’t always clearly describe the data they store (semantics). And two schemas can often be connected in multiple ways. Therefore, it is preferable to have the real-world context documented in some way so it can be referred to, e.g. a thesaurus containing enforced company standards, domain language, etc.

The proposed solution

The authors propose a schema matching component called Cupid, which should be able to perform general-purpose schema matching on different data sources by only considering schema information and not instance data (schema based).

Cupid tackles the problem by calculating multiple similarity scores between schema elements and deriving a mapping from those scores. It does this by representing schemas as trees with the schema elements being the leaf nodes. Cupid performs the following phases:

Phase 1 Linguistic matching; pairs of schema elements are matched based on their names, data types, and domains resulting in a linguistic similarity coefficient ($lsim$). This coefficient has a value in the range of [0,1] of which 1 is considered a perfect score. It has the following steps:

1. Normalization: We perform tokenization, expansion, and elimination. A thesaurus is used to look up general vocabulary and domain specific terms.

2. Categorization: Schema elements are clustered into categories based on their data types, schema hierarchy, and linguistic content. Elements can belong to multiple categories.
3. Comparison: $lsim$ is computed by performing sub-string matching on the tokens originating from the name synonym, or hypernymy (thesaurus) of the schema elements.

Phase 2 Structural matching; for every pair of schema elements, we calculate the similarity of their respective contexts. This results in the structural similarity ($ssim$), which also is a value between [0,1]. Ancestor similarity directly impacts this score; highly similar ancestors elevate it while low similarity ancestors lower it. This is configurable using parameters.

Phase 3 Weighted similarity & mapping; using $lsim$ and $ssim$ the weighted similarity is calculated as follows: $wsim = w_{struct} \times ssim + (1 - w_{struct}) \times lsim$, where w_{struct} is in the range [0,1]. Final mappings are created by selecting pairs of schema elements with the highest (based on threshold) weighted similarity values. This can result in a 1:1 or $1{:}\,n$ cardinality.

Any optimization techniques or special cases discussed

Among the paper's optimization techniques, lazy expansion and pruning stands out for me as computational improvements. Pruning limits the tree size and depth, whereas lazy expansion prevents duplicate comparisons by comparing elements in the graph before tree conversion.

As a special case, Cupid interprets referential constraints as potential join view. For each foreign key, we introduce a node that represents the join of the participating tables. This will have a positive effect on structural similarity but can cause the tree to grow a lot extra.

The strengths and limitations of Cupid

A major strength of Cupid is that it works on many different data models. It does this by changing (non-hierarchical) schemas into a graph first and then into a tree on which the phases are executed. Another nice part about using trees is that it allows for context dependent mapping during the structural matching (basically including the parent node).

Schema tree construction fails if a schema contains a cyclic relationship as it turns into an infinite recursion. Cupid is also dependent on a thesaurus for linguistic matching, if dropped the results seem to be worse or unchanged.

The evaluation methodology

The authors evaluated Cupid against two other schema-based prototypes, DIKE and MOMIS, by comparing their performance on canonical examples that isolate specific matching properties and real-world examples. A quantitative comparison of Cupid, DIKE, and MOMIS is not possible because schema matching is subjective, and Dike and MOMIS were designed for another use case (schema integration). The performance was therefore assessed qualitatively.

Experiment Results

For this paper I was unable to find the source code used by the authors to run their qualitive experiments. There is a Python package called Valentine, which implements Cupid as one of their schema matching algorithms, but it has the flaw that it only accepts tabular datasets in the form of a Pandas or Polars frame. Therefore, I have decided to create the code myself together with the help of GitHub Copilot, using the extended paper as a reference.

The code together with the original paper has been made public on my GitHub account: https://github.com/OssamaSijbesma/generic-schema-matching-with-cupid

Parameter Settings

For the experiment we use synthetic datasets that exactly match the schema from the paper. Since the algorithm is schema-based, the results mirror those from the original datasets. These datasets are organized into subfolders by experiment within the *dataset* folder on GitHub.

The adjustable parameters used for the algorithm are a one-on-one match from the paper:

```
"th_accept": 0.5,  # Acceptance similarity threshold for the final mapping, in [0, 1]
"th_high": 0.6,  # High-confidence threshold used during structural propagation, in [0, 1]
"th_low": 0.35,  # Low-confidence threshold used during structural propagation, in [0, 1]
"c_inc": 1.2,  # Positive reinforcement coefficient for matching children nodes, in [0, +inf)
"c_dec": 0.9,  # Negative reinforcement coefficient for non-matching children nodes, in [0, +inf)
"w_struct": 0.6,  # Weight of structural similarity in the weighted score, in [0, 1]
```

Figure 2, Parameters used for the experiment

The authors note using a positive reinforcement value of 0.5 for identical type matches within the range [0,0.5]. However, they do not specify the value used for other type combinations, I have therefore selected the value of 0.2 as a default to these non-identical combinations. A matrix with a value for every type combination could serve as a solution for future implementation.

Reproduced results

As mentioned before, schema matching is inherently subjective, requiring a qualitative evaluation of the results. Furthermore, comparing algorithms designed for different use cases provides no value and is omitted in the reproduction. Instead, this evaluation focuses on reproducing the six canonical examples (EXPT 1–6) and one real life example (EXPT 7) established by the authors. Because these tests yield clear pass/fail outcomes, they allow for a direct comparison with the original published results.

To perform these evaluations, I developed a test script that takes a source and target dataset, runs the algorithm with the specified settings, and displays all resulting matches with their associated scores (see Figure 3).

EXPT 1 [PASS]: Identical schemas. Two identical schemas were presented to the algorithm which resulted in a perfect match.

EXPT 2 [PASS]: Atomic elements with same names, but different data types. We expose two almost identical schemas of which one element has a different type. The result is still a perfect match but the score of the element with the changed type is lower.

```
Source Schema:
  Customer
    Customer_Number : integer
    Name : string
    Address : string

Target Schema:
  Customer
    CustomerNumber : integer
    CustomerName : string
    StreetAddress : string

3 Cupid matches:
  Customer.Customer_Number <-> Customer.CustomerNumber: 0.76
  Customer.Name <-> Customer.CustomerName: 0.627
  Customer.Address <-> Customer.StreetAddress: 0.627
```

Figure 3, Results of EXPT 3

EXPT 3 [PASS]: Atomic elements with same data types, but different names. We add a prefix and remove an underscore as shown in Figure 3. The result is a perfect match; the elements with a prefix scored lower than the one with the removed underscore. This displays how much a name match actually matters.

EXPT 4 [PASS]: Different class names. One class is called *Person* the other is called *Customer*, the elements have the same names and data types. Nonetheless the results have a perfect match.

EXPT 5 [PASS]: Different Nesting of the data. The schemas are similar but one has a nested structure and the other one is flat. Interestingly enough the score was higher for the nested elements than for the non-nested ones, due to nested nodes being able to get more reinforcement. We also notice a 1:m cardinality, *FirstName* matches with *FirstName* but also *LastName*.

EXPT 6 [PASS]: Type Substitution or Context dependent mapping. One schema has two *Address* elements; the other schema has *BillTo* and *ShipTo*. The Address matched with both *BillTo* and *ShipTo*, but because of its parent class it was able to correctly distinguish the two.

EXPT 7 [PASS]: CIDX vs. Excel. This real-world example involves two highly different schemas. We were able to verify the few matches the authors mention, but only with the author provided domain-specific thesaurus. Furthermore, this is the first case where we clearly see that not all elements can or will be matched, which is fine when there are no similar elements.

Discussion

By implementing Cupid's algorithm, we were able to reproduce the paper's results. The canonical examples were a great help to easily validate each test case. However, I found that the parameter thresholds needed to be set quite low to successfully pass all cases, which ultimately highlights a trade-off between sensitivity and specificity. On the technical side, turning schemas into trees to perform linguistic and structural comparisons is very clever as it allows for many different scenarios and formats. However, strong real-world performance heavily depends on a quality thesaurus. Regarding performance, I believe incorporating value range metadata could be used to further indicate similarity between two values (e.g. two strings with a length from up to 4 characters). To end, because Cupid is schema-based, it allows large privacy sensitive organizations like banks to perform schema matching on large scale.